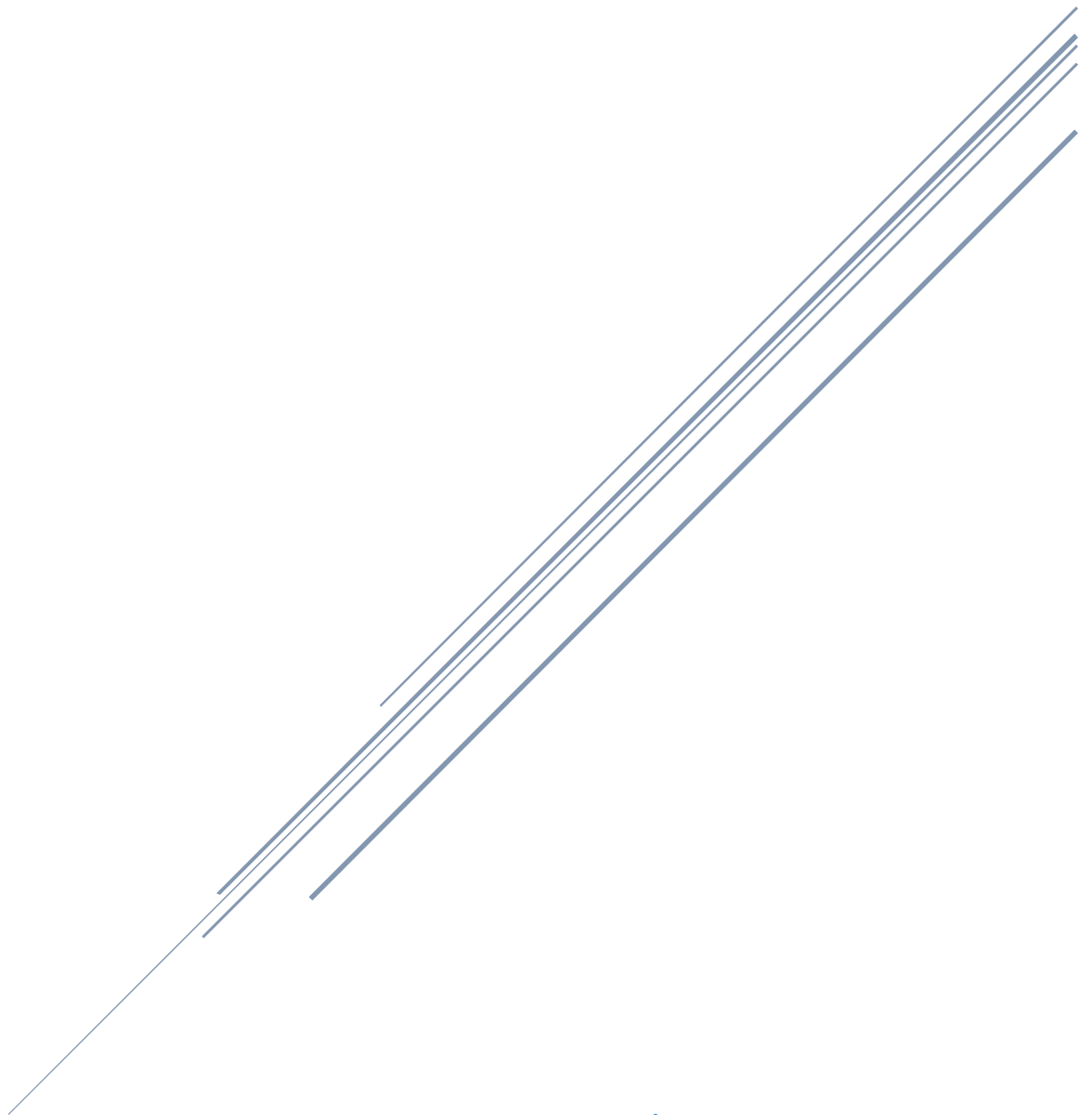


CODE AND SUMMARY OF CLASS 9

Sahir Ahmed Sheikh

Saturday (2 – 5)



Teachers:

Muhammad Bilal And Ali Aftab Sheikh

Code And Summary Of Class 9 – Saturday (2 – 5) | Quarter 3

Introduction

This document provides a detailed summary of Class 9 of Quarter 3, conducted on Saturday from 2-5 PM. The session was led by Sir Ali Aftab Sheikh, with Sir Muhammad Bilal. The class aimed to revise and deepen the understanding of Object-Oriented Programming (OOP) concepts, specifically focusing on the four pillars: **Polymorphism, Inheritance, Abstraction, and Encapsulation**. The session combined theoretical discussions, real-world analogies, coding examples, and motivational advice to prepare students for upcoming tests and Quarter 4 challenges.

The class addressed the following key objectives:

- Revisiting OOP concepts with practical coding examples.
- Clarifying misconceptions about course materials (e.g., Sir Arif's slides).
- Emphasizing the importance of active participation, practice, and preparation for tests.
- Preparing students for Quarter 4, which will introduce more advanced content.

Students were reminded of the limited classes remaining in Quarter 3 (two to three sessions), the upcoming test within the same week, and the immediate start of Quarter 4 in June without a long gap. The session also included light-hearted discussions about current events (e.g., drone memes) to keep the atmosphere engaging, alongside announcements about Class Representative (CR) selections and assignments.

Topics Covered

1. Class Updates and Quarter 3 Context

- **Quarter 3 Overview:** The class highlighted that Quarter 3 is nearing its end, with only two to three classes remaining. The environment in Quarter 3 was described as relatively relaxed, with Python concepts being manageable. However, Quarter 4 was introduced as significantly more challenging, requiring students to start preparing immediately.
- **Test Updates:** The previous test was reviewed as easy, covering content directly from class materials. The next quiz was announced for May, with a final test scheduled within the week after the last Quarter 3 class. Students were urged to study Sir Arif's slides and notebook for theoretical questions, as the class summaries are derived from these resources.
- **CR Announcement:** A Class Representative (CR) was to be announced to guide students on study content and provide additional resources for those interested in studying beyond class. Unfortunately, no boys were selected as CRs, with only four girls chosen, prompting a call for boys to step up.
- **Motivational Advice:** Students were encouraged to stay focused, avoid procrastination, and engage actively. The importance of skills over degrees in the IT industry was reiterated, with warnings against copy-pasting code and the need to understand logic for professional settings.

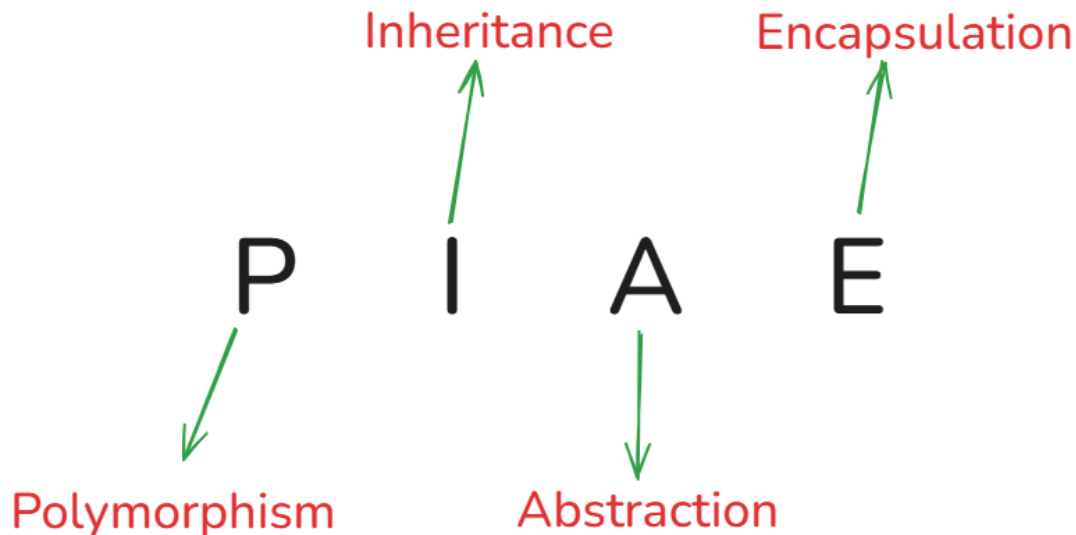
*2. Revision of OOP Concepts**:*

The session began with a revision of OOP concepts, using relatable analogies (e.g., biryani recipe, family business) to reinforce understanding. A blueprint analogy was used to explain classes and objects:

- **Class:** A blueprint or template defining attributes (variables) and methods (functions). For example, a biryani recipe class defines ingredients and cooking steps.
- **Object:** An instance of a class, created from the blueprint. Using the biryani class, each batch of biryani is an object with the same structure but potentially different values (e.g., spice levels).

- **Key Insight:** Classes ensure consistency in structure across objects, while objects can have varying attribute values. This was demonstrated with a house blueprint analogy, where every house follows the same design but may differ in details like paint color.

3. Four Pillars of OOP (PIAE Mnemonic)



The class focused on the four pillars of OOP, remembered using the mnemonic **PIAE** (Polymorphism, Inheritance, Abstraction, Encapsulation), inspired by “Pakistan International Airlines + E.” Each pillar was explained with definitions, real-world analogies, coding examples, and practical applications.

3.1 Polymorphism

- **Definition:** Polymorphism (“many forms”) allows objects of different classes to be treated as objects of a common parent class, with each object behaving uniquely through method overriding. It enables flexibility by allowing a single method name to produce different outcomes based on the object calling it.

- **Real-World Analogy:** A remote control’s “play” button plays a movie on a DVD player and a song on a music player, demonstrating the same action with different outcomes.
- **Types:**
 - **Compile-time (Static):** Method overloading (less common in Python).
 - **Run-time (Dynamic):** Method overriding (common in Python via inheritance).
- **Benefits:** Simplifies code, enhances reusability, and supports scalability by allowing a single interface for varied behaviors.
- **Coding Example:**

```
class BankAccount:
    def send_notification(self):
        return f"Hello {self.account_holder}, your bank balance is {self._balance}"

class StudentAccount(BankAccount):
    def send_notification(self): # Override parent method
        return f"Hello {self.account_holder}, your pocket money is {self._balance}"

# Create objects
account = BankAccount("Ali", 1000)
student = StudentAccount("Ahmed", 500)

# Demonstrate polymorphism
print(account.send_notification()) # Output: Hello Ali, your bank balance is 1000
print(student.send_notification()) # Output: Hello Ahmed, your pocket money is 500
```

- **Explanation:** The `send_notification` method is overridden in `StudentAccount` to provide a different message, showcasing polymorphism. The same method name produces different outputs based on the object type.

3.2 Inheritance

- **Definition:** Inheritance allows a child class to inherit attributes and methods from a parent class, promoting code reuse and hierarchical organization. It supports “is-a” relationships (e.g., a `SavingsAccount` is a `BankAccount`).

- **Real-World Analogy:** A family recipe passed down through generations, where each generation inherits and may modify the recipe.
- **Types:**
 - Single Inheritance: One parent, one child.
 - Multiple Inheritance: One child, multiple parents.
 - Multilevel Inheritance: Grandparent → Parent → Child.
 - Hierarchical Inheritance: One parent, multiple children.
- **Benefits:** Reduces code duplication, supports extensibility, and models real-world hierarchies.
- **Coding Example:**

```
class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            return f"Deposited {amount}. New balance: {self._balance}"
        return "Invalid deposit amount"

class SavingsAccount(BankAccount):
    def __init__(self, account_holder, balance, interest_rate):
        super().__init__(account_holder, balance) # Inherit parent's constructor
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self._balance * self.interest_rate
        self._balance += interest
        return f"Added interest {interest}. New balance: {self._balance}"

# Create objects
savings = SavingsAccount("Ali", 1000, 0.05)
print(savings.deposit(500)) # Output: Deposited 500. New balance: 1500
print(savings.add_interest()) # Output: Added interest 75.0. New balance: 1575
```

- **Explanation:** `SavingsAccount` inherits `deposit` from `BankAccount` and adds `add_interest`, demonstrating code reuse and extension. The `super()` function calls the parent's constructor to initialize inherited attributes.

3.3 Abstraction

- **Definition:** Abstraction hides complex implementation details and exposes only necessary parts through a simplified interface. In Python, it is achieved using abstract base classes (ABCs) via the `abc` module.
- **Real-World Analogy:** Using a smartphone's interface (e.g., camera, volume buttons) without understanding its internal circuits.
- **Benefits:** Reduces complexity, enforces consistency across subclasses, and minimizes errors by hiding unnecessary details.
- **Coding Example:**

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

# Create objects
circle = Circle(5)
rectangle = Rectangle(4, 6)
print(circle.area()) # Output: 78.53981633974483
print(rectangle.area()) # Output: 24
```

- **Explanation:** `Shape` is an abstract class with an abstract `area` method, enforced by `@abstractmethod`. Child classes (`Circle`, `Rectangle`) must implement `area`, ensuring a consistent interface while hiding implementation details.

3.4 Encapsulation

- **Definition:** Encapsulation bundles data and methods into a class, restricting direct access to attributes using access modifiers (public, protected, private) to protect the object's internal state.
- **Real-World Analogy:** A bank account allows deposits and withdrawals through specific methods but hides the balance to prevent unauthorized changes.
- **Python's Approach:**
 - **Public:** Accessible everywhere (e.g., `account_holder`).
 - **Protected:** Indicated by `_attribute` (convention, not enforced).
 - **Private:** Indicated by `__attribute` (name mangling, restricts access).
- **Benefits:** Ensures data integrity, supports modularity, and allows controlled access with validation.
- **Coding Example:**

```
class BankAccount:
    def __init__(self, account_holder, initial_balance):
        self.account_holder = account_holder # Public
        self.__balance = initial_balance # Private

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return f"Deposited {amount}. New balance: {self.__balance}"
        return "Invalid deposit amount"

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return f"Withdrew {amount}. New balance: {self.__balance}"
        return "Invalid or insufficient funds"

# Create object
account = BankAccount("Ali", 1000)
print(account.account_holder) # Output: Ali
print(account.get_balance()) # Output: 1000
print(account.deposit(500)) # Output: Deposited 500. New balance: 1500
print(account.withdraw(200)) # Output: Withdrew 200. New balance: 1300
print(account.withdraw(2000)) # Output: Invalid or insufficient funds
# print(account.__balance) # AttributeError: No attribute __balance
```

- **Explanation:** The `__balance` attribute is private, accessible only through `get_balance`, `deposit`, and `withdraw`. Validation in methods ensures safe modifications, and direct access to `__balance` raises an error, demonstrating data protection.

4. Practical Coding and Debugging

- **Google Colab:** Used for coding exercises, with students encouraged to familiarize themselves with its interface.
- **Debugging Example:** The class addressed common errors, such as missing constructor arguments in inherited classes (e.g., `StudentAccount` needing `account_holder` and `balance`). Students were guided to create proper constructors using `super().__init__`.
- **Validation Logic:** Conditions were added to `deposit` and `withdraw` methods to prevent invalid transactions (e.g., depositing zero or withdrawing more than the balance), reinforcing the importance of robust code.

5. Key Insights and Takeaways

- **OOP's Power:** The four pillars enable modular, reusable, and maintainable code, turning programming into a creative and structured process.
- **Skills Over Degrees:** The IT industry prioritizes problem-solving and practical skills, making consistent practice essential.
- **Avoid Copy-Pasting:** Understanding code logic is critical for professional settings, as copying without comprehension leads to errors.
- **Test Preparation:** Upcoming tests are crucial, with potential course discontinuation for failures. Students must review all topics, including those not covered in class due to time constraints.
- **Quarter 4 Preparation:** Quarter 4 will introduce advanced topics (e.g., AI Agents), requiring proactive learning and engagement.

Class Resources & Assignments

Today's Class Code

- The code for Class 9 is available on Google Colab: [Class Code](#)
- Students are encouraged to experiment with the provided examples (e.g., depositing/withdrawing small amounts, overriding methods) to deepen understanding.

Assignments

1. Build_Compose_and_Decorate_A_Complete_Traditional_OOP:

- Work on the 06_Build_Compose_and_Decorate_A_Complete_Traditional_OOP project from the GitHub repository:
[Build Compose and Decorate A Complete Traditional OOP – GitHub Repo](#)
- Apply OOP concepts (e.g., polymorphism, inheritance, encapsulation, abstraction) to build a complete traditional OOP system.
- **Deadline:** Before the next class.
- **Submission:** Upload to LinkedIn and submit via the [Assignment Submission Form](#).

2. Complete Previous Assignments:

- Ensure all prior assignments are completed, shared on LinkedIn, and submitted via the class submission form.
- Review feedback to improve code quality and understanding.
- **Deadline:** Before the next class.

Important Reminder

- As announced in class, a Class Representative (CR) and a team for Quarter 4 have been formed. To join the team, complete all assignments before the next class. The CR will review submissions and assign roles accordingly.
- The next class is the second-last of Quarter 3, and the formed team will continue into Quarter 4. Prioritize completing assignments to secure your role.
- Use Sir Arif's slides and notebook for theoretical preparation, as they form the basis of test questions.
- Stay consistent, ask questions, and engage actively to master OOP, which is foundational for Quarter 4 and industry projects.

Final Words

The responsibility now lies with you. Learn actively, participate confidently (even wrong answers aid learning), and practice logic daily. The upcoming test is critical, and Quarter 4 will demand greater effort. Embrace challenges, complete assignments, and prepare for advanced topics like AI Agents. Stay focused to seize opportunities in the IT industry.

Allah Hafiz – See you in the next class!

Repo Link: [GIAIC-Sat-Afternoon](#)

This is my repository where I have uploaded all the class codes, assignments, and detailed summaries. You can go through each class's code and its full detailed summary without any hesitation, and it will also help you prepare well for your quiz.

Thank You for Reading!

Hope you understood Class 9 well.

"Education is the kindling of a flame, not the filling of a vessel." –
Socrates